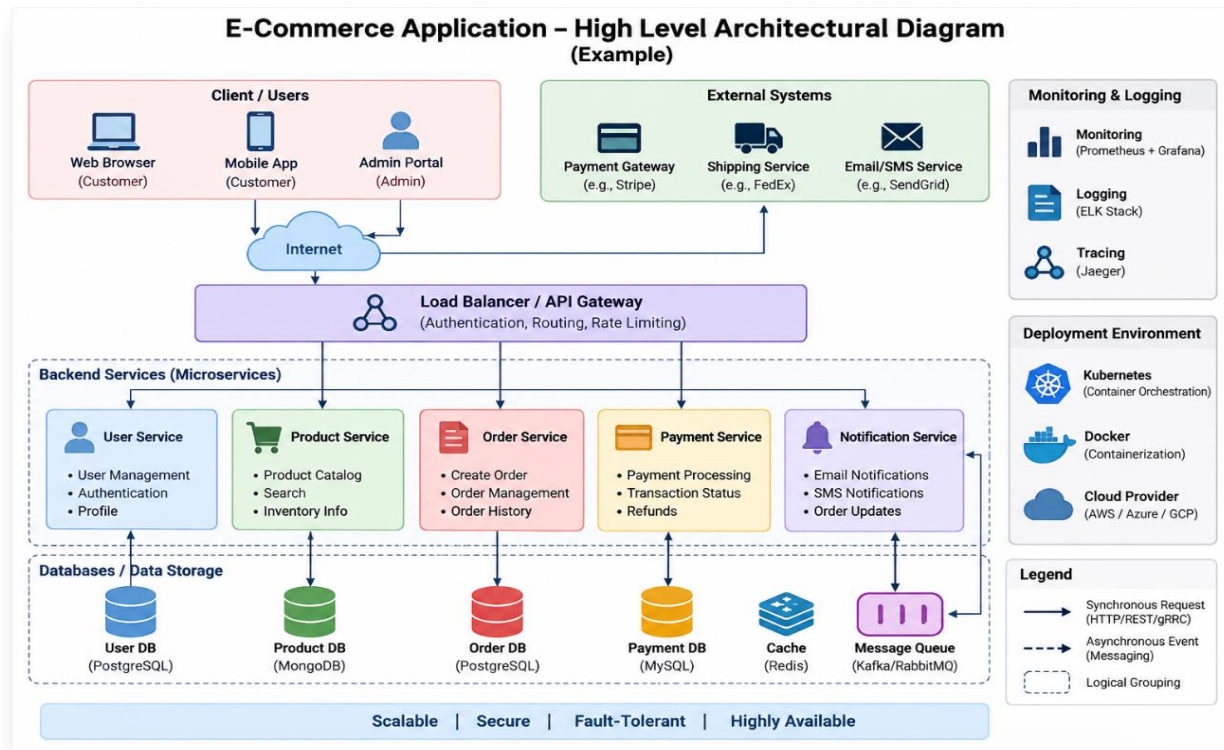




How to Understand the Architectural Diagram of Any Project — Step by Step

An **architectural diagram** is a visual representation of how a software system is structured, how its major components interact, and how data or requests move through the system.

Think of it as a **map of a software project**. Before reading source code, an architecture diagram helps you understand *what exists*, *why it exists*, and *how everything connects*.

1. First Understand What “Software Architecture” Means

Definition

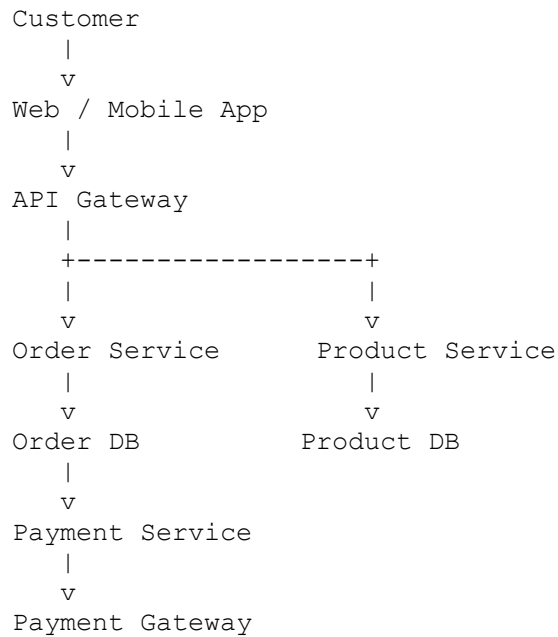
Software architecture is the high-level organization of a software system, including:

- Components/modules
- Responsibilities of components
- Communication between components
- Data flow
- External systems
- Databases
- Infrastructure
- Security boundaries

- Deployment structure
- Technology choices

Simple example

Imagine an e-commerce application:



You should not immediately start asking “*What programming language is this written in?*”

First ask:

What is this system trying to accomplish, and what are the major parts involved?

2. Understand the Purpose of the Diagram

Before reading individual boxes, determine **why the diagram was created**.

Different architectural diagrams answer different questions.

Diagram type	Main question answered
System Context Diagram	Who/what interacts with the system?
Container Diagram	What major applications/services make up the system?
Component Diagram	What components exist inside an application/service?

Class Diagram	What classes and relationships exist?
Sequence Diagram	In what order do interactions occur?
Deployment Diagram	Where does the software run?
Data Flow Diagram	How does data move?
Network Diagram	How are systems/network components connected?
Cloud Architecture	How are cloud services organized?
Database ER Diagram	How is data structured?
Infrastructure Diagram	What infrastructure is required?

Example

If you see:

```
User → Web Application → API → Database
```

you are probably looking at a **high-level system architecture**.

If you see:

```
UserService
|
+-- UserController
+-- UserService
+-- UserRepository
```

you are looking at a more detailed **application/component architecture**.

3. Identify the Diagram's Scope

This is one of the most important steps.

Ask:

What part of the project does this diagram cover?

A diagram could represent:

Entire Organization
↓
Entire Product
↓
Application
↓
Microservice
↓
Module
↓
Class
↓
Method

Example

Suppose a project contains:

```
E-Commerce Platform
├── Web Application
├── Mobile Application
├── API Gateway
├── Order Service
├── Payment Service
├── Inventory Service
├── Notification Service
└── Databases
```

A diagram showing only:

```
Order Service
|
+-- Order Controller
+-- Order Service
+-- Order Repository
```

does **not** represent the entire e-commerce architecture.

It represents the **internal architecture of Order Service**.

4. Find the Starting Point

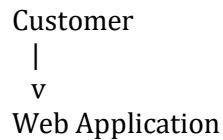
Look for the component that initiates interaction.

Usually it is:

- User
- Customer

- Administrator
- Mobile application
- Web browser
- External system
- Scheduled job
- Message/event

For example:



The **Customer** is the actor.

The **Web Application** is the entry point into the system.

Questions to ask

1. Who starts the process?
2. Where does the request originate?
3. What is the first software component receiving it?
4. Is the interaction synchronous or asynchronous?

5. Identify Every Box

Now go through the diagram and identify each box.

For every box, ask:

What is this component?

A box might represent:

- Application
- Service
- Module
- Database
- Queue
- API
- Server
- Cloud service
- External system
- User
- Device
- Load balancer

Create a small table while reading.

Component	Type	Responsibility
Web App	Application	User interface
API Gateway	Gateway	Routes requests
Order Service	Microservice	Handles orders
Payment Service	Service	Processes payments
Order DB	Database	Stores orders
Payment Gateway	External system	Processes payment

This technique makes complicated diagrams much easier.

6. Understand Each Component's Responsibility

Knowing the component's name isn't enough.

You need to determine:

What job does this component perform?

For example:

API Gateway

Definition: A gateway that acts as an entry point between clients and backend services.

Possible responsibilities:

```
Client
  ↓
API Gateway
  ├── Authentication
  ├── Routing
  ├── Rate limiting
  └── Request forwarding
```

Order Service

Responsible for:

Create Order
Cancel Order
Update Order
Get Order
Calculate Order Total

Database

Responsible for persistent storage:

Orders
Customers
Products
Payments

7. Understand the Arrows

Arrows are often more important than boxes.

An arrow usually represents communication or dependency.

Example:

A —————> B

could mean:

A sends a request to B.

But you must inspect the arrow label.

For example:

```
Web App
  |
  | HTTPS/REST
  v
API Gateway
```

This tells you:

- Communication direction
 - Protocol
 - Possible API style
-

8. Determine the Direction of Data Flow

Follow arrows carefully.

Example:

```
Customer
  |
  | Order Request
  v
Web Application
  |
  | POST /orders
  v
Order Service
  |
  | SQL
  v
Order Database
```

The flow is:

```
Customer
  ↓
Web Application
  ↓
Order Service
  ↓
Database
```

But the response usually travels back:

```
Database
  ↓
Order Service
  ↓
Web Application
  ↓
Customer
```

Therefore, distinguish between:

Request flow

Client → Server

and

Response flow

Server → Client

9. Identify Communication Mechanisms

Look for labels such as:

- REST
- HTTP
- HTTPS
- gRPC
- GraphQL
- WebSocket
- TCP
- UDP
- JDBC
- SQL
- Kafka
- RabbitMQ
- AMQP
- MQTT

Example:

```
Order Service
      |
      | REST/HTTPS
      v
Payment Service
```

means the services communicate through a REST-based HTTP interface.

Whereas:

```
Order Service
      |
      | Kafka Event
      v
Notification Service
```

suggests asynchronous/event-driven communication.

10. Understand Synchronous vs Asynchronous Communication

This is extremely important in modern architectures.

Synchronous

The caller waits for a response.

```
Order Service
  |
  | Request
  v
Payment Service
  |
  | Response
  v
Order Service
```

Conceptually:

Order Service: "Please process payment."

Payment Service: "Done."

Order Service continues.

Asynchronous

The caller doesn't necessarily wait.

```
Order Service
  |
  | OrderCreated Event
  v
Message Broker
  |
  +-----> Notification Service
  |
  +-----> Analytics Service
```

The producer publishes an event and consumers process it independently.

11. Identify Databases and Storage

Look for:

- PostgreSQL
- MySQL
- Oracle
- MongoDB
- Redis

- Elasticsearch
- S3/object storage
- File systems

Then ask:

What data does each storage system contain?

Example:

```
Order Service
  |
  v
PostgreSQL
  |
  +-- Orders
  +-- OrderItems
  +-- Customers
```

If Redis appears:

```
Application
  |
  +-----> PostgreSQL
  |
  +-----> Redis
```

ask:

Why does the application need two different storage technologies?

Possible explanation:

```
PostgreSQL → permanent data
Redis       → cache/session/temporary fast-access data
```

Don't assume this without confirming the diagram/documentation, though.

12. Identify External Systems

External systems are extremely important because your application usually depends on things outside its control.

Examples:

```
Your Application
  |
```

```

+----> Payment Provider
|
+----> Email Provider
|
+----> SMS Provider
|
+----> Identity Provider
|
+----> Shipping Provider

```

Ask:

1. Who owns this system?
2. Is it internal or external?
3. What API does it expose?
4. What happens if it becomes unavailable?
5. Is authentication required?
6. Is the communication synchronous or asynchronous?

13. Find System Boundaries

Architectural diagrams often use:

- Boxes
- Containers
- Dashed lines
- Different colors
- Zones
- VPCs
- Subnets
- Packages

to indicate boundaries.

Example:

```

+----- YOUR SYSTEM -----+
|
| Web App
| |
| API Gateway
| |
| Backend Services
|
+-----+
|
| v

```

The boundary tells you:

Which components belong to your system and which do not.

14. Identify Layers

Many systems use layers.

A common architecture is:

```
Presentation Layer
    ↓
Application Layer
    ↓
Business Layer
    ↓
Data Access Layer
    ↓
Database
```

Example

```
Controller
    ↓
Service
    ↓
Repository
    ↓
Database
```

In a Java application:

```
UserController
    ↓
UserService
    ↓
UserRepository
    ↓
MySQL
```

Responsibilities

Layer	Responsibility
Controller	Receives requests

Service	Business logic
Repository	Data access
Database	Persistent storage

15. Identify Architectural Style

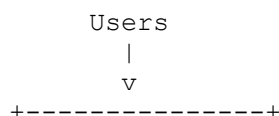
Once you understand the components, determine the overall architectural pattern.

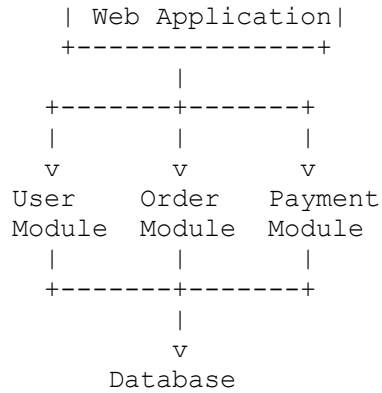
Common examples:

Architecture	Basic idea
Monolithic	One major deployable application
Layered	System divided into layers
Client-Server	Client communicates with server
Microservices	Multiple independently deployable services
Event-Driven	Components communicate through events
SOA	Services provide reusable business capabilities
Serverless	Functions/services managed by cloud platform
Hexagonal	Business logic isolated from external adapters
Clean Architecture	Dependencies point toward core business logic
Pipe-and-Filter	Data passes through processing stages

16. Example: Understand a Monolithic Architecture

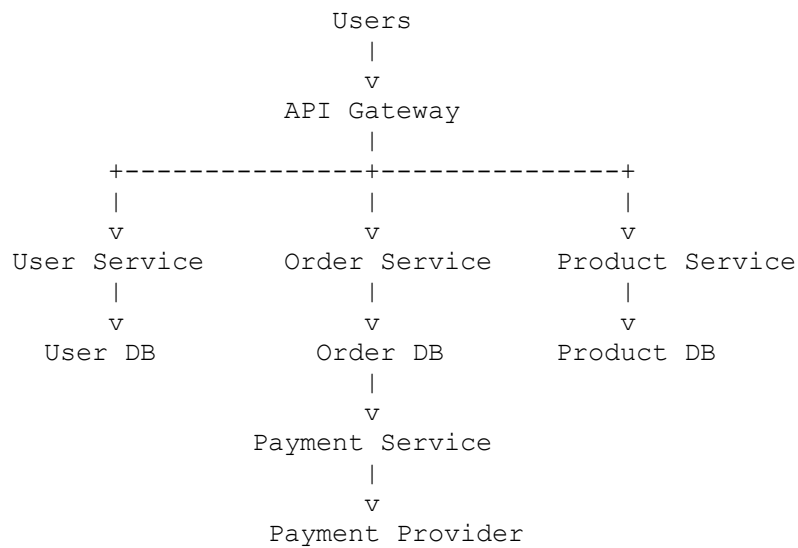
Suppose you see:





This suggests a **monolithic application** because the major business modules are contained within one application/deployment boundary.

17. Example: Understand Microservices Architecture



Read it step by step:

Step 1

Users interact with the API Gateway.

Step 2

The gateway routes requests.

Step 3

Different services handle different responsibilities.

Step 4

Services may have separate databases.

Step 5

Order Service communicates with Payment Service.

Step 6

Payment Service communicates with an external payment provider.

That gives you the **functional architecture**.

18. Then Understand Deployment Architecture

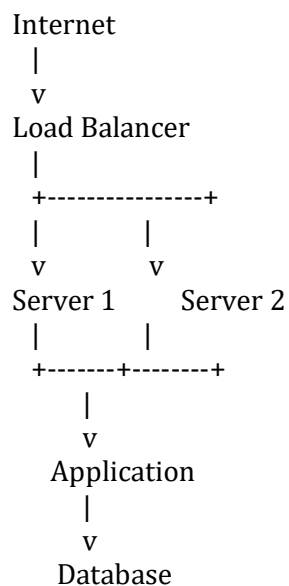
Functional architecture answers:

What does the software do?

Deployment architecture answers:

Where does the software run?

Example:

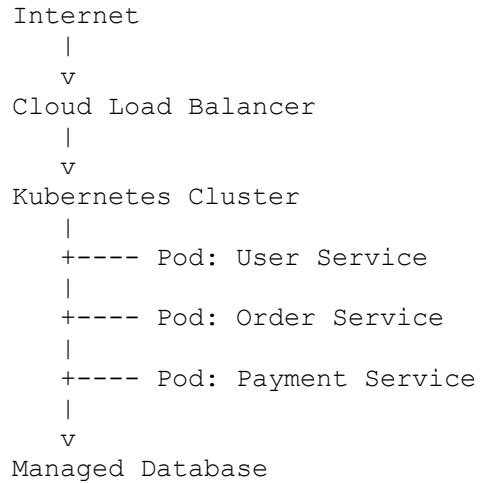


Now you are thinking about:

- Servers
 - Containers
 - Kubernetes
 - Cloud
 - Load balancing
 - Availability
 - Scaling
 - Networking
-

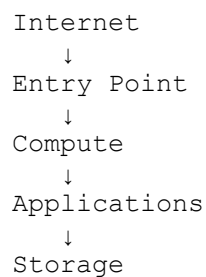
19. Understand Cloud Architecture

Suppose you see:



Don't try to understand every cloud service immediately.

Break it into layers:



Then study each technology.

20. Understand Security Boundaries

Look for:

- Firewall
- WAF
- API Gateway
- Authentication
- Authorization
- IAM
- OAuth
- JWT
- TLS
- Private network
- Public network
- VPN
- Security groups

Example:

```
Internet
  |
  v
WAF
  |
  v
API Gateway
  |
  v
Private Services
  |
  v
Private Database
```

This architecture suggests that the database isn't directly exposed to the public internet.

Ask:

Where does authentication happen?

Where does authorization happen?

Which components are public?

Which components are private?

21. Understand the Request Lifecycle

This is one of the best techniques for learning an unfamiliar architecture.

Take one real use case.

For example:

Customer places an order.

Then trace it through the entire architecture.

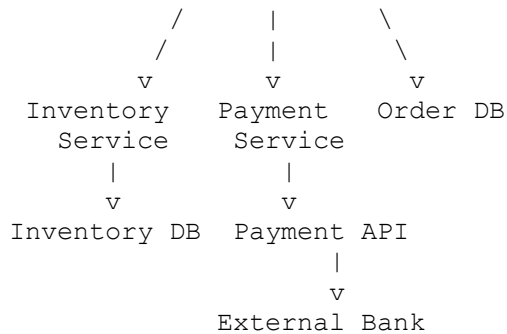
```
Customer
  ↓
Web App
  ↓
API Gateway
  ↓
Authentication
  ↓
Order Service
  ↓
Inventory Service
  ↓
Payment Service
  ↓
Order Database
  ↓
Message Broker
  ↓
Notification Service
  ↓
Email Provider
```

Now you've transformed a complicated diagram into a **story**.

22. Example: Complete Order Flow

Suppose the architecture is:

```
Customer
  |
  v
Web Browser
  |
  v
API Gateway
  |
  v
Order Service
```



Interpretation:

Step 1 — Customer

Customer initiates an order.

Step 2 — Browser

Browser sends the request.

Step 3 — API Gateway

Gateway receives and routes it.

Step 4 — Order Service

Order Service handles order business logic.

Step 5 — Inventory

Checks whether products are available.

Step 6 — Payment

Processes payment.

Step 7 — Database

Stores order information.

Step 8 — External bank

Payment provider communicates with an external financial system.

Now the diagram is no longer just boxes and arrows.

It represents a **business process**.

23. Ask “Why?” About Every Connection

Don't just read:

$A \rightarrow B$

Ask:

Why does A need B?

For example:

Order Service
|
v
Inventory Service

Why?

Possibly:

Order Service needs to verify inventory before confirming the order.

Then:

Order Service
|
v
Payment Service

Why?

The order requires payment authorization.

This turns architecture reading into **reasoning** rather than memorization.

24. Identify Dependencies

Create a dependency map.

Example:

Order Service
├── User Service
├── Inventory Service
├── Payment Service
├── Order Database
└── Message Broker

Then ask:

What happens if each dependency fails?

This leads naturally into reliability analysis.

25. Identify Single Points of Failure

A **Single Point of Failure (SPOF)** is a component whose failure can cause a critical part of the system to fail.

Example:

All Services
|
v
Single Database

If that database fails, many services may become unavailable.

A more resilient design might use:

Database
/ \
v v
Primary Replica

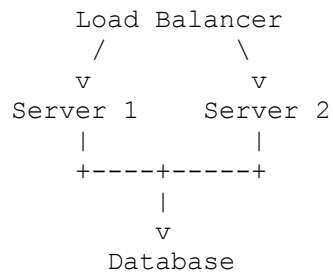
or another appropriate redundancy strategy.

26. Understand Scalability

Ask:

Which components need to handle increasing traffic?

Example:



This indicates horizontal scaling of application servers.

Then ask:

Can the database scale similarly?

That question often reveals architectural constraints.

27. Understand Performance

Look for:

- Cache
- CDN
- Load balancer
- Message queues
- Database indexes
- Replicas
- Batch processing
- Async processing

Example:

```
graph TD; User --> CDN; CDN --> WA[Web Application]; WA --> RC[Redis Cache]; RC --> DB[(Database)];
```

You should ask:

Why is Redis here?

Potential answer:

To reduce repeated database reads and improve response time.

Again, confirm the actual design intent rather than assuming it.

28. Understand Reliability

Look for:

```
Service A
|
+---- Service B
|
+---- Service C
```

Then ask:

- What happens if B fails?
- Is there retry logic?
- Is there a timeout?
- Is there a circuit breaker?
- Is the operation idempotent?
- Is there a fallback?
- Is data replicated?
- Is there disaster recovery?

Architecture diagrams sometimes explicitly show these mechanisms.

29. Understand Observability

Modern architecture often contains:

```
Applications
|
+---- Logs
|
+---- Metrics
|
+---- Traces
|
v
Observability Platform
```

Understand:

Logs

What happened?

Metrics

How much/how often?

Examples:

CPU usage
Request count
Error rate
Latency

Traces

Where did a particular request travel?

Example:

Gateway
↓
Order Service
↓
Payment Service
↓
Database

Tracing lets you follow that request across components.

30. Identify Technology Choices

Only **after understanding the architecture** should you examine technologies.

Example:

Frontend	→ React
Backend	→ Java/Spring Boot
Database	→ PostgreSQL
Cache	→ Redis
Messaging	→ Kafka
Deployment	→ Kubernetes
Cloud	→ AWS

Create a technology map:

Area	Technology	Purpose
Frontend	React	User interface
Backend	Spring Boot	Business APIs
Database	PostgreSQL	Persistent data
Cache	Redis	Fast temporary access
Messaging	Kafka	Event communication
Container	Docker	Packaging
Orchestration	Kubernetes	Container management
Cloud	AWS	Infrastructure

31. Connect Architecture to Source Code

Once the diagram makes sense, map it to the repository.

For example:

```

Architecture
  ↓
Order Service
  ↓
Repository
  ↓
order-service/

```

Then:

```

order-service/
├── controller/
│   └── OrderController.java
├── service/
│   └── OrderService.java
├── repository/
│   └── OrderRepository.java
├── model/
│   └── Order.java

```

└─ config/

Now the architecture diagram becomes a guide for navigating the codebase.

32. Architecture → Code Mapping

A useful mapping technique is:

Diagram element	Possible code representation
Service	Application/module
API	Controller/endpoint
Business logic	Service class
Database	Repository/DAO
Entity	Model/entity class
Message queue	Producer/consumer
External API	Client/adaptor
Authentication	Security module/filter
Cache	Cache repository/service
Configuration	Config files/classes

This isn't universal, but it is a useful starting point.

33. Understand the Data Model

If the architecture contains a database, find out:

What data enters?

↓

Where is it stored?

↓

Who reads it?

↓

Who updates it?

↓

Who deletes it?

Example:

```
Order Service
  |
  v
orders
  |
  +---- order_id
  +---- customer_id
  +---- order_date
  +---- status
  +---- total
```

Then determine relationships:

```
Customer
  |
  +---- Orders
        |
        +---- Order Items
              |
              +---- Products
```

34. Understand Error Paths

Most beginners follow only the successful path.

Don't.

Look for:

```
Request
  |
  v
Payment
  |
  +---- Success → Order Confirmed
  |
  +---- Failure → Retry
                  |
                  +---- Failed → Cancel Order
```

Ask:

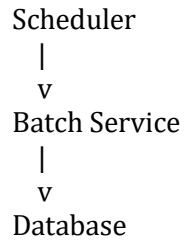
What happens when something goes wrong?

This often reveals more about an architecture than the happy path.

35. Understand Batch and Scheduled Processes

Not every process starts with a user.

You may see:



Example:

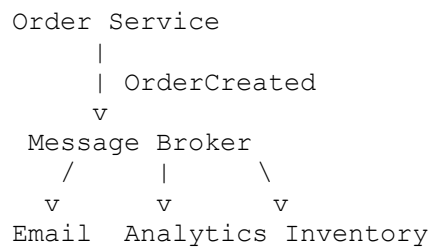
Every night, the system generates invoices.

This is a different flow from:

User → Web → API

36. Understand Event-Driven Architecture

Example:



Interpretation:

- Order Service produces an event.
- Broker stores/distributes the event.
- Multiple consumers react independently.

Ask:

Who produces the event?

Who consumes it?

What happens if a consumer is temporarily unavailable?

37. Understand the Architecture at Multiple Levels

Don't try to understand everything at once.

Use **zoom levels**.

Level 1 — System

Users
↓
Application
↓
External Systems

Level 2 — Containers/services

Frontend
↓
Gateway
↓
Services
↓
Databases

Level 3 — Components

Controller
↓
Service
↓
Repository

Level 4 — Code

Class
↓
Method
↓
Statement

A very effective rule is:

Understand the higher level before moving to the lower level.

38. Use the “5 Questions” Method

For every component, ask:

Question	Example
What?	What is Order Service?
Why?	Why does it exist?
Input?	What does it receive?
Output?	What does it produce?
Dependency?	What does it depend on?

Example:

Order Service

What?

Handles orders.

Why?

Provides order-related business functionality.

Input?

Order requests/events.

Output?

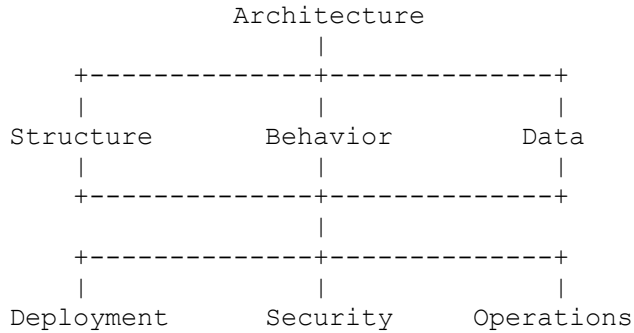
Order responses/events.

Dependencies?

Inventory, payment, database, etc.

39. Use the “6 Dimensions” Method

When studying any architecture, analyze these six dimensions:



1. Structure

What components exist?

2. Behavior

How do they interact?

3. Data

How does data move and persist?

4. Deployment

Where does everything run?

5. Security

Who can access what?

6. Operations

How is the system monitored, scaled and maintained?

40. A Complete Step-by-Step Architecture Reading Process

Use this sequence whenever someone gives you an unfamiliar architecture diagram.

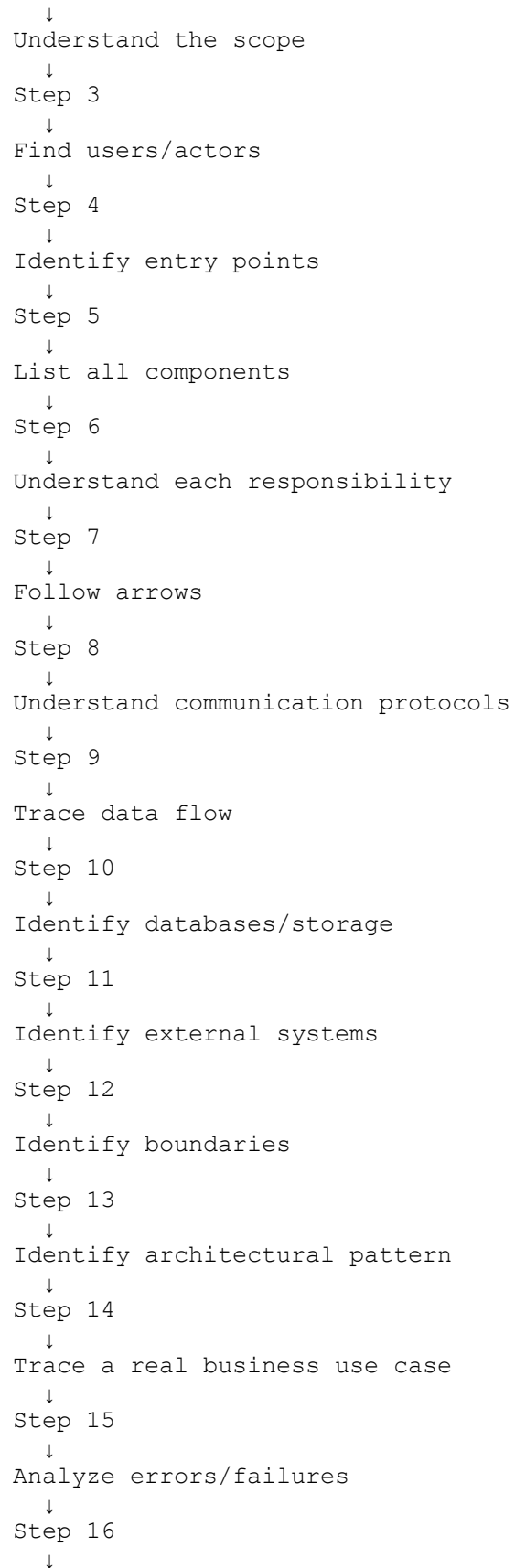
Step 1

↓

Identify the diagram type

↓

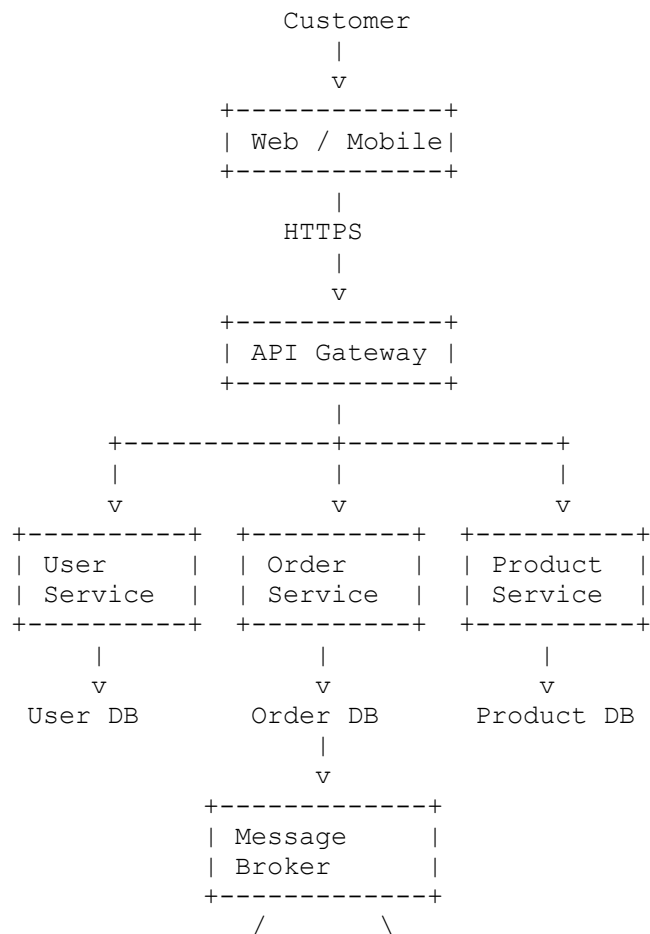
Step 2

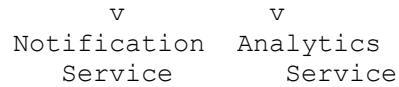


Analyze security
 ↓
 Step 17
 ↓
 Analyze scalability/performance
 ↓
 Step 18
 ↓
 Analyze deployment
 ↓
 Step 19
 ↓
 Map architecture to source code
 ↓
 Step 20
 ↓
 Explain the entire system in your own words

41. Practical Example

Consider this architecture:





Now analyze it.

A. Who uses the system?

Customer

B. What are the clients?

Web / Mobile

C. What's the entry point?

API Gateway

D. What are the major services?

User Service
Order Service
Product Service
Notification Service
Analytics Service

E. What databases exist?

User DB
Order DB
Product DB

F. What communication style exists?

There is:

HTTPS

between client and gateway.

There is also:

Message Broker

for asynchronous communication.

G. What happens when an order is created?

Potentially:



You now understand the architecture without needing to know every line of code.

42. Architecture Review Checklist

When you receive an architecture diagram, use this checklist.

Business

- ☐ What problem does the system solve?
- ☐ Who uses it?
- ☐ What are the major use cases?

Structure

- ☐ What are the major components?
- ☐ What does each component do?
- ☐ What are the system boundaries?

Communication

- ☐ Which components communicate?
- ☐ Which direction does communication flow?
- ☐ REST, gRPC, messaging, etc.?

- ☐ Synchronous or asynchronous?

Data

- ☐ Where is data stored?
- ☐ Which service owns which data?
- ☐ How does data move?
- ☐ Is caching involved?

Security

- ☐ Where is authentication?
- ☐ Where is authorization?
- ☐ What is public?
- ☐ What is private?
- ☐ Where is encryption used?

Reliability

- ☐ What happens if a service fails?
- ☐ Are there retries?
- ☐ Are there timeouts?
- ☐ Are there redundant components?
- ☐ Are there single points of failure?

Performance

- ☐ Where are caches?
- ☐ Where are queues?
- ☐ Where can scaling occur?
- ☐ Are databases bottlenecks?

Deployment

- ☐ Where does each component run?
- ☐ VM, container, Kubernetes, serverless?
- ☐ Which cloud/platform?
- ☐ How are components deployed?

Operations

- ☐ Where are logs?
 - ☐ Where are metrics?
 - ☐ Where are traces?
 - ☐ How are alerts generated?
-

43. The Most Important Mental Model

When you see an architecture diagram, don't think:

"I have to understand all these boxes."

Instead think:

"I need to understand the story of a request moving through the system."

For example:

WHO?

↓

Customer

WHAT?

↓

Places an order

WHERE?

↓

Web/Mobile App

HOW?

↓

HTTPS → API Gateway

WHICH SERVICE?

↓

Order Service

WHAT DATA?



Order Database

WHAT DEPENDENCIES?



Inventory + Payment

WHAT HAPPENS NEXT?



OrderCreated Event

WHO CONSUMES IT?



Notification + Analytics

WHERE DOES IT RUN?



Cloud/Kubernetes/etc.

Once you can answer those questions, the architecture becomes much easier to understand.

44. A Simple Formula to Remember

Use:

A → C → R → D → E → F → S

Where:

- **A — Actors:** Who uses the system?
- **C — Components:** What are the major parts?
- **R — Relationships:** How are they connected?
- **D — Data:** What data moves/stores where?
- **E — Execution:** Where does it run?
- **F — Failure:** What happens when something breaks?
- **S — Security:** Who can access what?

Then add:

T — Trace a real use case.

So your final architecture-reading formula becomes:

Actors → Components → Relationships → Data → Execution → Failure → Security → Trace

That method works well for **monoliths, microservices, Java/Spring applications, cloud systems, APIs, event-driven systems, enterprise applications, and distributed systems.**

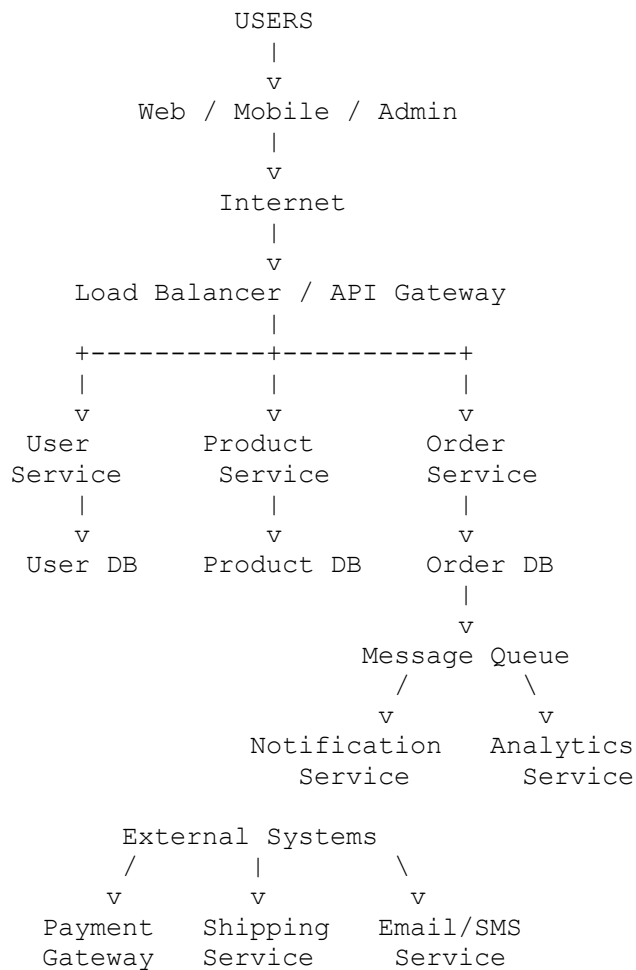
Detailed Explanation of the E-Commerce Architectural Diagram

The diagram represents a **high-level microservices architecture for an e-commerce application**. It shows how customers and administrators interact with the system, how requests reach backend services, how services store data, how asynchronous messaging works, and how external systems, deployment, monitoring, and logging fit together.

The easiest way to understand it is to read it **from top to bottom and then from left to right**.

1. Overall Architecture at a Glance

The complete system can be simplified to:



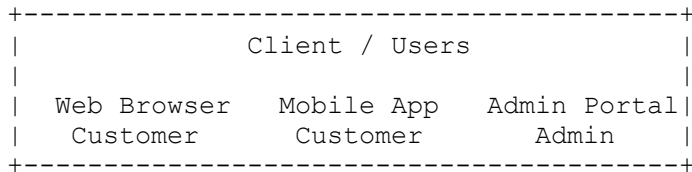
The major architectural layers in the image are:

1. **Client / Users**
2. **Internet**

3. **Load Balancer / API Gateway**
4. **Backend Microservices**
5. **Databases / Data Storage**
6. **External Systems**
7. **Messaging**
8. **Monitoring & Logging**
9. **Deployment Environment**
10. **Architecture qualities**

2. Client / Users

The top-left section is:

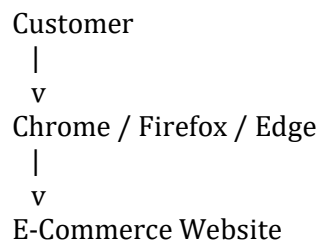


This represents the **people and applications that initiate interaction with the e-commerce system.**

2.1 Web Browser

The web browser is the interface used by customers.

For example:



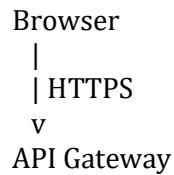
A customer might:

- Register
- Login
- Search products
- Add products to cart
- Place orders

- Make payments
- View order history

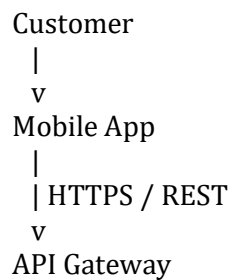
The browser communicates with backend APIs.

Typically:



3. Mobile Application

The second client is the mobile application.



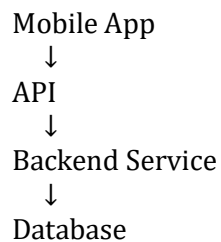
The mobile app could be Android or iOS.

The important architectural concept is that the mobile application generally **doesn't directly access databases**.

Incorrect:

Mobile App → Database

Instead:



This protects the database and keeps business logic on the server.

4. Admin Portal

The third client is:

```
Admin Portal
|
v
API Gateway
```

The administrator may perform operations such as:

- Manage products
- Manage users
- View orders
- Process refunds
- Monitor inventory
- Manage users
- View reports

The admin portal normally requires stronger authorization than an ordinary customer application.

For example:

Customer → Customer APIs

Administrator → Administrative APIs

5. Internet

The diagram contains an **Internet** cloud between clients and the backend.

Conceptually:

```
Web Browser
  \
Mobile App ----> Internet
  /
Admin Portal
  |
  v
  API Gateway
```

The Internet is the communication network through which external clients reach the application.

Typically communication is protected using:

HTTPS

rather than plain HTTP.

6. Load Balancer / API Gateway

This is one of the most important components in the architecture.

The diagram shows:

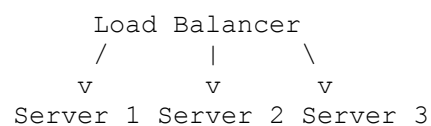


It performs several possible responsibilities.

6.1 Load Balancer

A **load balancer** distributes incoming traffic among multiple backend instances.

For example:



Suppose 10,000 customers are accessing the application.

Instead of sending every request to one server:

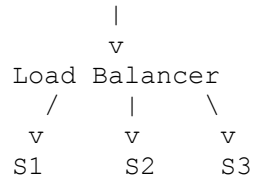
10,000 requests



Server 1

requests can be distributed:

10,000 requests



This improves scalability and availability.

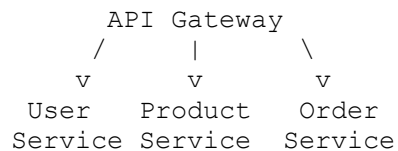
7. API Gateway

An **API Gateway** is a central entry point for API requests.

Instead of:

Mobile → User Service
Mobile → Product Service
Mobile → Order Service
Mobile → Payment Service

the client can use:



The gateway can handle:

- Routing
 - Authentication
 - Authorization
 - Rate limiting
 - Request validation
 - API versioning
 - Sometimes logging
 - Sometimes request transformation
-

8. Authentication

The diagram explicitly mentions authentication.

Authentication means:

Determining who the user is.

For example:

```
User
↓
Login
↓
Username + Password
↓
Authentication System
↓
Access Token
```

The token might then be sent with future requests.

Conceptually:

Authorization: Bearer <token>

9. Rate Limiting

Rate limiting controls how many requests a client can make within a period.

For example:

User → API Gateway

The gateway might enforce a policy such as:

100 requests / minute

This can help protect backend services from excessive traffic.

10. Backend Services — Microservices

The large middle section is:

```
+-----+
|           Backend Services (Microservices)           |
|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
| User | Product | Order | Payment | Notification |   |   |
+-----+
```

This is the heart of the application.

Each service represents a **business capability**.

11. What Is a Microservice?

A **microservice** is a relatively small, independently deployable application/service focused on a specific business capability.

Instead of creating one enormous application:

```
+-----+
|      E-Commerce Application      |
|                                  |
| User + Product + Order + Payment |
| + Notification + Everything Else  |
+-----+
```

the architecture separates responsibilities:

User Service
Product Service
Order Service
Payment Service
Notification Service

This is called **service decomposition**.

12. User Service

The first backend service is:

```
+-----+
| User Service      |
|                  |
| User Management   |
| Authentication    |
| Profile           |
+-----+
```

Its responsibility is managing user-related functionality.

Possible operations:

Create User
Update User

Get User
Delete User
Login
Logout
Manage Profile

Example:

Customer
|
| Login
v
API Gateway
|
v
User Service
|
v
User DB

13. User Database

The diagram connects User Service to:

User DB
(PostgreSQL)

This database stores user-related information.

Possible conceptual data:

USER

user_id
name
email
password_hash
phone
status
created_at

The important principle is:

User Service owns user-related data.

14. Product Service

The second service is:

+-----+		
	Product Service	
	Product Catalog	
	Search	
	Inventory Info	
+-----+		

Its responsibilities include:

Product catalog

```
Product
├── ID
├── Name
├── Description
├── Price
└── Category
```

Search

Customers can search for products.

Example:

```
"laptop"
  ↓
Product Service
  ↓
Search products
  ↓
Return results
```

Inventory information

The service can provide information about product availability.

15. Product Database

The diagram shows:

Product DB
(MongoDB)

This represents storage for product-related data.

For example:

Products
Categories
Product Attributes
Inventory Information

MongoDB is shown here as an example; the architecture does not inherently require MongoDB.

16. Order Service

The central business service is:

```
+-----+
| Order Service |
|               |
| Create Order  |
| Order Management |
| Order History |
+-----+
```

This handles order-related operations.

For example:

```
Customer
|
| Place Order
v
Order Service
```

The service may:

1. Receive order request.
 2. Validate order.
 3. Check product information.
 4. Check inventory.
 5. Initiate payment.
 6. Create order.
 7. Store order.
 8. Publish an order event.
 9. Trigger notifications.
-

17. Order Database

The diagram shows:

Order DB
(PostgreSQL)

Possible information:

```
ORDER
-----
order_id
customer_id
order_date
status
total_amount
payment_status
shipping_address
```

It might also have related tables such as:

```
Order
 |
+-- OrderItem
     |
     +-- Product
```

18. Payment Service

The diagram shows:

```
+-----+
| Payment Service      |
|                      |
| Payment Processing   |
| Transaction Status   |
| Refunds              |
+-----+
```

This service is responsible for payment-related business operations.

Possible flow:

```
Order Service
 |
 | Payment Request
 v
Payment Service
 |
```

The Payment Service can handle:

- Payment initiation
- Payment status
- Payment confirmation
- Refunds
- Transaction records

19. Payment Database

The diagram shows:

Payment DB
(MySQL)

It can store information such as:

Transaction ID
Order ID
Payment Status
Amount
Currency
Payment Method Reference
Created Time

Important: Sensitive payment credentials should not normally be stored directly by the application unless there is a specific compliant architecture for doing so.

20. Notification Service

The fifth service is:

```
+-----+
| Notification Service |
|                     |
| Email Notifications  |
| SMS Notifications    |
| Order Updates        |
+-----+
```

Its responsibility is communicating information to users.

Examples:

Order confirmed
Payment successful
Order shipped
Order delivered
Password reset

Instead of making the Order Service directly send every email, the architecture can separate notification responsibility.

21. Why Have a Separate Notification Service?

Consider:

Order Service
|
v
Email Provider

The Order Service now depends directly on the email provider.

A more decoupled architecture can use:

Order Service
|
v
Message Queue
|
v
Notification Service
|
v
Email Provider

Now notification processing is separated from order processing.

22. Databases / Data Storage Layer

The diagram groups databases under:

```
+-----+  
|      Databases / Data Storage      |  
+-----+
```

It contains:

User DB
Product DB
Order DB
Payment DB
Cache
Message Queue

Strictly speaking, **cache and message queue are not databases in the same sense as PostgreSQL/MySQL**, but they are data/storage or infrastructure components, so grouping them here is understandable in a high-level diagram.

23. Why Separate Databases?

The architecture uses:

User Service → User DB

Product Service → Product DB

Order Service → Order DB

Payment Service → Payment DB

This is commonly associated with the **database-per-service pattern**.

The idea is that each service has ownership over its data.

Conceptually:

User Service
|
v
User DB

Order Service
|
v
Order DB

rather than:

User Service ----+
Order Service ----+----> One giant database
Payment Service -+

However, database-per-service is a design choice, not a mandatory requirement for microservices.

24. Redis Cache

The diagram contains:

Cache
(Redis)

A **cache** stores frequently accessed information so it can be retrieved faster.

Without caching:

```
Application
|
v
Database
```

With caching:

```
Application
|
v
Redis
|
| Cache miss
v
Database
```

Example:

```
Customer requests popular product
|
v
Redis
|
Cache Hit
|
v
Response
```

This can reduce database load and improve latency.

25. Message Queue

The diagram contains:

Message Queue
(Kafka / RabbitMQ)

This is used for **asynchronous communication**.

Instead of:

Order Service
|
| Wait
v
Notification Service

you can have:

Order Service
|
| Event
v
Message Queue
|
v
Notification Service

The producer and consumer become more loosely coupled.

26. Kafka vs RabbitMQ

The image says:

Kafka / RabbitMQ

These technologies are not identical, but both can support messaging architectures.

Kafka

Often used for:

- Event streaming
- High-throughput event pipelines
- Distributed event processing
- Analytics pipelines

RabbitMQ

Often used for:

- Message queues
- Task distribution
- Routing messages
- Asynchronous processing

The actual choice depends on project requirements.

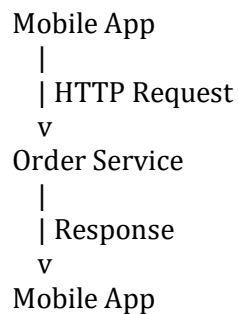
27. Synchronous Communication

The legend shows a solid arrow:

—————>
Synchronous Request
(HTTP/REST/gRPC)

Synchronous means the caller generally waits for the response.

Example:



Example:

GET /products/123

The application expects a response.

28. Asynchronous Communication

The diagram's dashed arrow represents:

- - - - ->
Asynchronous Event
(Messaging)

For example:

```
Order Service
  |
  | OrderCreated
  v
Message Queue
  |
  v
Notification Service
```

The Order Service does not necessarily wait for Notification Service to finish.

This provides **looser coupling**.

29. Logical Grouping

The dashed boxes in the diagram represent **logical boundaries/groups**.

For example:

```
+-----+
| Backend Services                |
|                                |
| User | Product | Order | Payment |
+-----+
```

This doesn't necessarily mean that all those services run on one physical server.

It simply communicates:

These components belong to the backend-service layer of the architecture.

30. External Systems

The upper-middle section is:

```
+-----+
| External Systems                |
|                                |
| Payment | Shipping | Email/SMS |
+-----+
```

These systems are outside the core e-commerce application.

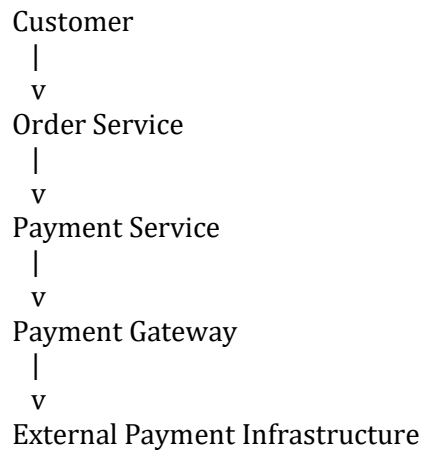
31. Payment Gateway

The diagram gives an example:

Payment Gateway
(e.g., Stripe)

The application might communicate with an external payment provider.

Flow:



The application should generally treat this as an external dependency.

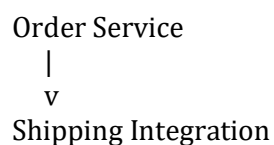
32. Shipping Service

The diagram gives:

Shipping Service
(e.g., FedEx)

The e-commerce system might send shipping information to an external shipping provider.

Example:



|
v
Shipping Provider

Possible operations:

Create Shipment
Get Tracking Number
Get Shipping Status
Calculate Shipping

33. Email/SMS Service

The diagram gives:

Email/SMS Service
(e.g., SendGrid)

The Notification Service may use an external provider.

For example:

Notification Service
|
v
Email/SMS Provider
|
v
Customer

This allows the application to outsource message delivery infrastructure.

34. Monitoring & Logging

The right-hand panel contains:

Monitoring
Logging
Tracing

These are collectively part of **observability**.

35. Monitoring

The diagram gives:

Prometheus + Grafana

Monitoring answers questions such as:

Is the application healthy?

How much traffic are we receiving?

How many requests are failing?

What is the latency?

Typical metrics include:

CPU Usage

Memory Usage

Request Count

Error Rate

Response Time

Database Connections

36. Prometheus

Prometheus is commonly used to collect and store metrics.

Conceptually:

Services

|

| Metrics

v

Prometheus

|

v

Metrics Storage

37. Grafana

Grafana can visualize metrics in dashboards.

Conceptually:

Prometheus

|

v

Grafana

|

v

Dashboard

A dashboard might show:

Requests/sec

Error rate

CPU

Memory

Latency

38. Logging

The diagram shows:

Logging

(ELK Stack)

Logs are records of events occurring inside applications.

Example:

2026-09-15 10:15:20

OrderService

Order 12345 created

Logs are useful for troubleshooting.

39. ELK Stack

ELK commonly refers to:

Elasticsearch

Logstash

Kibana

A simplified flow:

Applications

|

v

Logs

|

v

Logstash

|

v

Elasticsearch

|

v

Kibana

Kibana can then be used to search and visualize logs.

40. Distributed Tracing

The diagram shows:

Tracing

(Jaeger)

Tracing helps answer:

What happened to one request as it travelled through multiple services?

For example:

Request ID: ABC123

API Gateway

↓ 10 ms

Order Service

↓ 50 ms

Payment Service

↓ 200 ms

Payment Gateway

↓

Response

You can identify where the request spent most of its time.

This is especially useful in microservices.

41. Deployment Environment

The right side shows:

Kubernetes
Docker
Cloud Provider

These describe where/how the application can run.

42. Docker

Docker packages an application and its dependencies into a container.

Conceptually:

```
Application
+
Dependencies
+
Runtime
  ↓
  Container
```

For example:

```
Order Service
  ↓
Docker Image
  ↓
Docker Container
```

43. Kubernetes

Kubernetes is a **container orchestration platform**.

Instead of manually managing hundreds of containers:

```
Container 1
Container 2
Container 3
...
Container 100
```

Kubernetes can help manage:

- Deployment
- Scaling
- Service discovery
- Health checks
- Restarting failed containers
- Rolling updates

Conceptually:

```
Kubernetes Cluster
|
+-- User Service
+-- Product Service
+-- Order Service
+-- Payment Service
+-- Notification Service
```

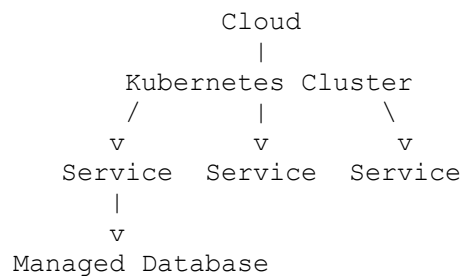
44. Cloud Provider

The diagram lists:

AWS / Azure / GCP

These are examples of cloud platforms.

A real deployment could look conceptually like:



The image intentionally treats the cloud provider generically.

45. Complete Request Flow

Now let's combine everything.

Suppose a customer places an order.

Step 1 — Customer

Customer

decides to purchase a product.

Step 2 — Mobile/Web Application

Customer



Web Browser / Mobile App

Step 3 — Internet

Web/Mobile



Internet

Step 4 — API Gateway

Internet



Load Balancer / API Gateway

The gateway can authenticate and route the request.

Step 5 — Order Service

API Gateway



Order Service

Step 6 — Product/Inventory

The Order Service may need product/inventory information.

Order Service



Product Service

Step 7 — Payment

Order Service



Payment Service
↓
Payment Gateway

Step 8 — Store Order

Order Service
↓
Order DB

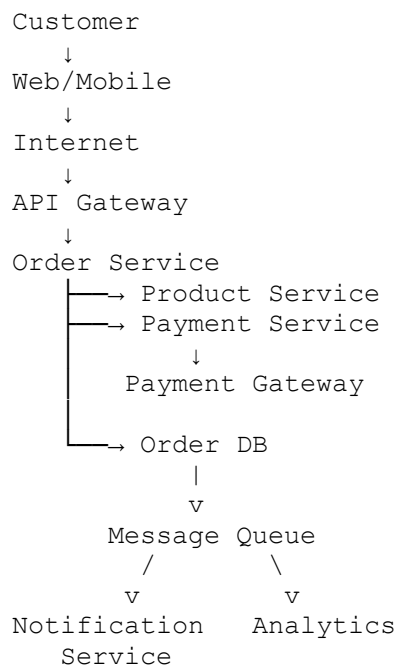
Step 9 — Publish Event

Order Service
|
| OrderCreated
v
Message Queue

Step 10 — Notification

Message Queue
↓
Notification Service
↓
Email/SMS Provider
↓
Customer

The entire flow becomes:



|
v
Email/SMS

46. What Happens If Payment Fails?

This is where architectural thinking becomes important.

Imagine:

Order
↓
Payment Service
↓
Payment Gateway
↓
FAILED

The system might then:

Payment Failed
↓
Order marked FAILED/PAYMENT_PENDING
↓
Customer informed

The exact behavior depends on the application's business rules.

The important architectural question is:

How does the system handle failure?

47. What Happens If Notification Service Is Down?

Suppose:

Order Service
↓
Message Queue
↓
Notification Service
X
unavailable

The queue can potentially retain the event until the consumer becomes available, depending on the messaging system and configuration.

This illustrates an advantage of asynchronous architecture:

Order Processing

≠

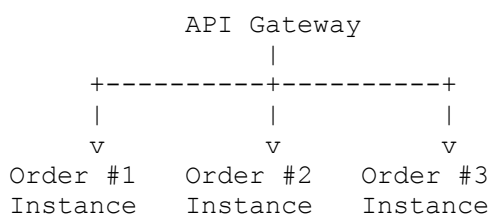
Notification Processing

They can be partially decoupled.

48. What Happens If Order Service Has High Traffic?

Suppose thousands of customers place orders simultaneously.

The system can potentially scale Order Service:



A container orchestrator such as Kubernetes can help manage multiple instances.

49. Why the Architecture Is Called Scalable

The bottom of the image says:

Scalable | Secure | Fault-Tolerant | Highly Available

These are **architectural goals**, not automatic guarantees.

Scalable

The system can increase capacity as demand grows.

Secure

The system is designed to protect:

- Users

- Data
- APIs
- Infrastructure

Fault-tolerant

The system can continue operating when certain components fail.

Highly available

The system aims to minimize downtime.

Important: Simply drawing these words at the bottom does not prove that the architecture actually achieves them. They need to be demonstrated by concrete design decisions.

50. Security Interpretation

From the diagram, security-related elements include:

Client
↓
HTTPS
↓
API Gateway
↓
Authentication
↓
Backend Services
↓
Databases

A complete production architecture would normally need additional security controls, such as:

WAF
Identity Provider
Secrets Management
Encryption
Network Segmentation
Authorization
Audit Logging
Database Security

These aren't all explicitly shown in this example diagram.

51. How to Read the Arrows

There are three major visual concepts in the legend.

Solid arrow



Means:

Synchronous request/communication.

Examples:

Mobile → API Gateway
API Gateway → Order Service
Order Service → Database

Dashed arrow



Means:

Asynchronous event/message.

Example:

```
Order Service
      |
      | OrderCreated
      v
Message Queue
```

Dashed boundary

```
+ - - - - - - - +
| Logical Group   |
+ - - - - - - - +
```

Means:

These components are grouped logically.

It doesn't necessarily mean they are physically located together.

52. Architectural Relationships

We can create a relationship table from the diagram.

From	To	Purpose
Web Browser	Internet	User access
Mobile App	Internet	User access
Admin Portal	Internet	Administrative access
Internet	API Gateway	API access
API Gateway	User Service	User requests
API Gateway	Product Service	Product requests
API Gateway	Order Service	Order requests
API Gateway	Payment Service	Payment requests
API Gateway	Notification Service	Notification-related requests
User Service	User DB	User data
Product Service	Product DB	Product data
Order Service	Order DB	Order data
Payment Service	Payment DB	Payment data
Services	Redis	Cached data
Services	Message Queue	Asynchronous messaging
Payment Service	Payment Gateway	Payment processing
Notification Service	Email/SMS Provider	Notifications
Order-related systems	Shipping Provider	Shipping operations

53. Component Responsibility Table

Component	Primary responsibility
Web Browser	Customer UI
Mobile App	Customer mobile UI
Admin Portal	Administrative UI
Internet	Network connectivity
Load Balancer	Distribute traffic
API Gateway	API entry point/routing
User Service	User management
Product Service	Product/catalog functionality
Order Service	Order management
Payment Service	Payment processing
Notification Service	User notifications
User DB	User data
Product DB	Product data
Order DB	Order data
Payment DB	Payment data
Redis	Caching
Message Queue	Async communication
Payment Gateway	External payment integration
Shipping Service	External shipping integration
Email/SMS Service	External messaging
Prometheus	Metrics collection
Grafana	Metrics visualization
ELK	Log collection/search/visualization
Jaeger	Distributed tracing

Docker	Containerization
Kubernetes	Container orchestration
Cloud	Infrastructure hosting

54. How the Architecture Maps to a Java Project

If this were a Java/Spring Boot project, it might look conceptually like:

```
ecommerce/
├── user-service/
│   ├── controller/
│   ├── service/
│   ├── repository/
│   ├── entity/
│   └── configuration/
├── product-service/
│   ├── controller/
│   ├── service/
│   ├── repository/
│   └── entity/
├── order-service/
│   ├── controller/
│   ├── service/
│   ├── repository/
│   └── entity/
├── payment-service/
│   ├── controller/
│   ├── service/
│   └── repository/
└── notification-service/
    ├── consumer/
    ├── service/
    └── provider/
```

For example:

```
Architecture
  ↓
Order Service
  ↓
OrderController
  ↓
OrderService
```

↓
OrderRepository
↓
Order DB

This is how you move from an **architecture diagram to actual source code**.

55. Architecture vs Implementation

This distinction is very important.

The diagram says:

Payment Service

That does **not** necessarily tell you:

Java
Spring Boot
Maven
PostgreSQL
REST

Those are implementation details.

Architecture tells you primarily:

What components exist and how they interact.

Implementation tells you:

Exactly how those components are built.

56. The Diagram's Architecture Pattern

The example primarily demonstrates:

Microservices architecture

Because it separates:

User Service
Product Service
Order Service

Payment Service
Notification Service

It also demonstrates elements of:

API Gateway pattern

Clients
↓
API Gateway
↓
Services

Database-per-service pattern

User → User DB
Order → Order DB
Product → Product DB
Payment → Payment DB

Event-driven architecture

Service
↓
Message Queue
↓
Consumer

Containerized deployment

Docker
↓
Kubernetes

Observability architecture

Metrics → Prometheus/Grafana
Logs → ELK
Traces → Jaeger

So this isn't just one architectural pattern—it is a **combination of architectural patterns and infrastructure practices**.

57. The Best Way to Understand This Diagram

Use this sequence:

1. Identify users
- ↓
2. Identify client applications
- ↓
3. Find the entry point
- ↓
4. Identify backend services
- ↓
5. Understand each service's responsibility
- ↓
6. Identify databases
- ↓
7. Follow request arrows
- ↓
8. Follow event/message arrows
- ↓
9. Identify external systems
- ↓
10. Understand cache
- ↓
11. Understand deployment
- ↓
12. Understand monitoring/logging/tracing
- ↓
13. Trace one business use case
- ↓
14. Analyze security
- ↓
15. Analyze failures
- ↓
16. Analyze scalability
- ↓
17. Map components to source code

58. One-Minute Explanation of the Entire Diagram

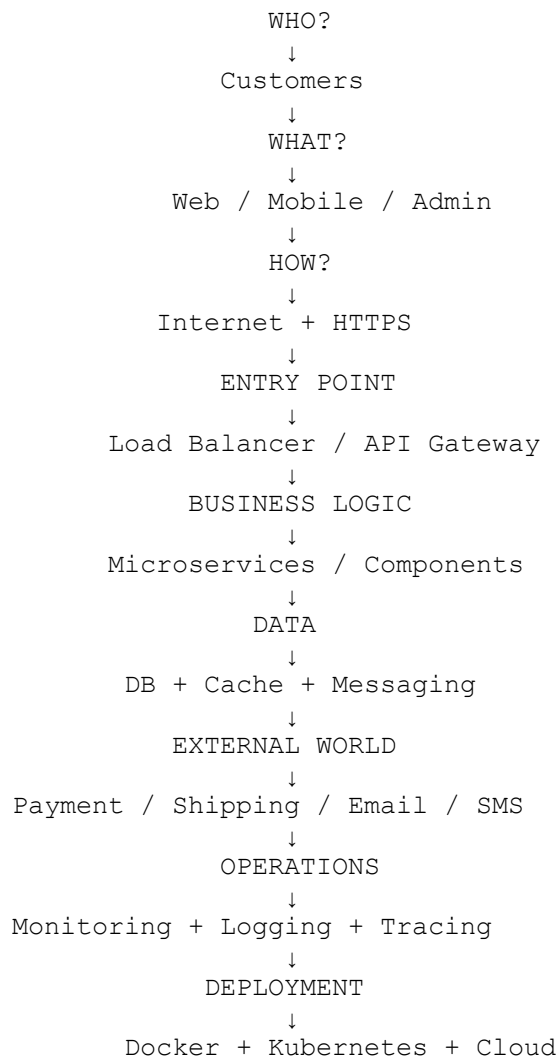
If you had to explain this architecture in an interview, you could say:

“This is a high-level e-commerce microservices architecture. Customers and administrators access the system through web, mobile, and admin clients over the Internet. Requests enter through a load balancer/API gateway, which handles concerns such as routing, authentication, and rate limiting. The gateway communicates with independently organized backend services such as User, Product, Order, Payment, and Notification services. Each major service has its own data store in this example. Redis provides caching, while Kafka or RabbitMQ provides asynchronous messaging. Payment, shipping, and email/SMS functionality can integrate with external providers. The services can be containerized with Docker and orchestrated using Kubernetes in a cloud environment. Prometheus/Grafana provide metrics and dashboards, ELK handles centralized logging, and Jaeger provides distributed tracing. The architecture aims for scalability, security, fault tolerance, and high availability.”

That paragraph is essentially the **entire diagram translated from visual form into architectural language**.

59. Final Mental Model

When you see an unfamiliar architecture diagram, remember:



The key skill is not memorizing the boxes. It is being able to trace one business operation—such as “customer places an order”—from the user interface all the way through the gateway, services, databases, external systems, messaging, and back to the user. Once you can do that, you genuinely understand the architecture rather than merely recognizing its components.